

打刻履歴・現在の打刻ステータス 取得用途API

ここからの実装は、先にバックエンドに関するものを一通り仕上げておき、そのうえでフロントエンドの実装に移るのが望ましいでしょう。

こうしておけば、フロントエンドの実装時に「動かないのは画面のせいかな、サーバーのせいかな？」と迷わずに済みます。

打刻履歴の取得API の実装

このAPIは、「誰が」「いつ」打刻したかというリストを、新しい順（降順）で取得するためのものです。

AttendanceRecordRepository.java への追記

まず、特定のユーザーの全記録を、日付が新しい順に並べて取得するメソッドを追加します。

- **ファイル名:** AttendanceRecordRepository.java

```
// 既存のメソッドの下に追加
java.util.List<AttendanceRecord> findByAccountIdOrderByClockInDesc(String accountId);
```

【解説】

`findByAccountId`（IDで探す）に `OrderByClockInDesc`（出勤時刻の降順で並べる）を組み合わせるだけで、最新の勤務日を一番上に持ってくるリストが取得できるようになります。

AttendanceRecordService.java への追記

サービス層には、リポジトリを呼び出すだけのシンプルなメソッドを追加します。

- **ファイル名:** AttendanceRecordService.java

```
// 特定のユーザーの全打刻履歴を取得する
public java.util.List<AttendanceRecord> getAllRecords(String accountId) {
    return repository.findByAccountIdOrderByClockInDesc(accountId);
}
```

AttendanceController.java への追記

最後に、フロントエンド（またはcurl）から「履歴をください」と言われた時に応える窓口を作ります。

- **ファイル名:** AttendanceController.java

```
/**
 * 打刻履歴一覧を取得する
 * GET http://localhost:8080/api/attendance/history?accountId=ユーザーID
 */
@GetMapping("/history")
public ResponseEntity<java.util.List<AttendanceRecord>> getHistory(@RequestParam String accountId) {
    java.util.List<AttendanceRecord> history = attendanceService.getAllRecords(accountId);
    return ResponseEntity.ok(history);
}
```

【解説：GETメソッドの使い分け】

これまではデータを「送る・作る」ための `@PostMapping` を使ってきましたが、今回はデータを「取得する」だけなので `@GetMapping` を使用します。

履歴取得のテスト (curl)

ファイルを保存して Spring Boot を再起動 (`./mvnw spring-boot:run`) したら、以下の順でテストしてみてください。

1. **ユーザー登録** (H2がリセットされるため) `curl -X POST -H "Content-Type: application/json" -d '{"userId":"test-user@example.com", "userName":"テスト", "password":"mysecretpassword"}' http://localhost:8080/api/users/register`
2. **出勤打刻** を1回行う
`curl -X POST "http://localhost:8080/api/attendance/clock-in?accountId=【メモしたaccountId】"`
3. **退勤打刻** を1回行う
`curl -X POST "http://localhost:8080/api/attendance/clock-out?accountId=【メモしたaccountId】"`
4. **履歴取得コマンド** を実行する

このうち末尾の4が今回実装した機能です。実装状況を確認するには以下のコマンドを実行します。

```
curl -X GET "http://localhost:8080/api/attendance/history?accountId=【登録時のaccountId】"
```

履歴取得が成功すると

以下のように、配列（`[ ]` で囲まれたリスト）の中に、打刻データが返ってきます。

```
[
  {
    "id": 1,
    "accountId": "...",
    "clockIn": "2026-05-11T12:00:00",
    "clockOut": "2026-05-11T12:05:00",
    "status": "退勤済み",
    "memo": null
  }
]
```

ステータス確認APIの実装

このAPIは、フロントエンドが最も必要とするものといえます。

ユーザーがアプリを開いたとき、画面に表示すべきボタンは、「出勤」か「退勤」のどちらか一方とする必要があるでしょう。

- **まだ出勤していない場合** → 「出勤ボタン」を表示
- **既に出勤している場合** → 「退勤ボタン」を表示

これをフロントエンド側で判断させるのではなく、バックエンドが「今の状態」を答えることで、画面側のロジックがシンプルになり、不正な操作も防げるのです。

AttendanceRecordService.javaへの追記

「現在の状態」を判定するロジックをサービスに追加します。

```
/**
 * 現在の打刻ステータスを取得する
 * @return "CLOCKED_IN" (出勤中), "CLOCKED_OUT" (退勤済み/未出勤)
 */
public String getCurrentStatus(String accountId) {
    return repository.findFirstByAccountIdOrderByIdDesc(accountId)
        .map(record -> {
            if (record.getClockOut() == null) {
                return "CLOCKED_IN";
            } else {
                return "CLOCKED_OUT";
            }
        })
        .orElse("CLOCKED_OUT"); // 記録が一つもない場合
}
```


【解説】

最新のレコードを取り出し、`clockOut`（退勤時刻）が `null` かどうかで状態を判定しています。
Javaの `Optional` の `map` と `orElse` を使うことで、データがない場合（初めての利用）もスマートに処理できます。

AttendanceController.java への追記

窓口を作成します。

```
/**
 * 現在のステータスを確認する
 * GET http://localhost:8080/api/attendance/status?accountId=ユーザーID
 */
@GetMapping("/status")
public ResponseEntity<String> getStatus(@RequestParam String accountId) {
    String status = attendanceService.getCurrentStatus(accountId);
    // 文字列をそのまま返すとJSONとして扱いにくいいため、シンプルなテキストで返します
    return ResponseEntity.ok(status);
}
```

SecurityConfig.java の確認

既に `/api/attendance/` を許可（`permitAll()`）しているはずですので、今回の `/status` もそのまま通るようになっているはずです。

ステータス確認のテスト (curl)

ファイルを保存して再起動したら、以下の流れで「状態が変わるか」を確認してください。

1. 初期状態の確認 (想定結果: CLOCKED_OUT)

```
curl -X GET "http://localhost:8080/api/attendance/status?accountId= 【accountId】 "
```

1. 出勤打刻をする (前回のPOSTコマンド)

2. もう一度ステータス確認 (想定結果: CLOCKED_IN)

```
curl -X GET "http://localhost:8080/api/attendance/status?accountId= 【accountId】 "
```

1. 退勤打刻をする

2. 最後にもう一度ステータス確認 (想定結果: CLOCKED_OUT)

```
curl -X GET "http://localhost:8080/api/attendance/status?accountId= 【accountId】 "
```

バックエンド通りの完成へ

お疲れ様でした！これで以下の機能が揃いました。

- ユーザー登録・ログイン
- 出勤・退勤の実行
- 打刻履歴の閲覧
- 現在の状態確認

勤怠管理アプリの「バックエンドAPI」としては、これで**最小構成**（MVP: Minimum Viable Product）が**完成**したと言えるでしょう。